```
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using DalApi;
using DO;
using System.Xml.Serialization;

namespace Dal;

internal class ProductImplementation : IProduct
{
    // בתיקיית הנתונים שלנו XML-הגדרתי פה את הנתיב לקובץ ה
    readonly string s_products_xml = @"..\xml\products.xml";

    // פונקציית עזר קטנה שחוסכת לי מלא כתיבה - היא פשוט טוענת את כל המוצרים מהקובץ
    private List<Product> LoadProducts()
    {
        // בתכלס, אם הקובץ לא נמצא (נניח בהרצה הראשונה), אני מחזירה רשימה ריקה ודהו
        if (!File.Exists(s_products_xml)) return new List<Product>();

        try
        {
            // C# לרשימה של XML-פה אני משתמשת בסריאלייזר בדי לתרגם את ה
            XmlSerializer serializer = new XmlSerializer(typeof(List<Product>), new XmlRootAttribute("ArrayOfProduct"));
            using (FileStream stream = new FileStream(s_products_xml, FileMode.Open))
            {
                // האמת שצריך לשים לב לסימן קריאה בסוף בדי שהקומפיילר לא יציק על null
                return (List<Product>)serializer.Deserialize(stream)!;
            }
        }
        catch { return new List<Product>(); }
    }

    // וזאת הפונקציה ההפוכה - היא פשוט דורסת את הקובץ הקיים עם הרשימה החדשה
    private void SaveProducts(List<Product> products)
    {
        XmlSerializer serializer = new XmlSerializer(typeof(List<Product>), new XmlRootAttribute("ArrayOfProduct"));
        using (FileStream stream = new FileStream(s_products_xml, FileMode.Create))
        {
            serializer.Serialize(stream, products);
        }
    }

    // --- מימוש ה-CRUD ---

```

```
    public int Create(Product item)
    {
        // אני שואבת את כל הרשימה הקיימת מה-XML
        List<Product> products = LoadProducts();

        // בודקת שאין כבר מוצר עם אותו ID, חבל שיהיו כפילויות שיבלבלו אותנו
        if (products.Any(p => p.id == item.id))
            throw new Exception($"Product with ID {item.id} already exists");

        // מוסיפה את הפריט החדש לרשימה ומעדכנת את הקובץ בבת אחת
        products.Add(item);
        SaveProducts(products);
        return item.id;
    }

    public Product? Read(int id)
    {
        // שקיבלנו ID-פשוט חיפוש מהיר ברשימה לפי ה
        return LoadProducts().FirstOrDefault(p => p.id == id);
    }

    public Product? Read(Func<Product, bool> filter)
    {
        // באן זה חיפוש גמיש יותר לפי מה שהמשתמש ביקש לסנן
        return LoadProducts().FirstOrDefault(filter);
    }

    public IEnumerable<Product?> ReadAll(Func<Product, bool>? filter = null)
    {
        // מביאה את כל המוצרים, ואם יש פילטר אז אני מסננת אותם בדרך
        List<Product> products = LoadProducts();
        if (filter == null) return products;
        return products.Where(filter);
    }

    public void Update(Product item)
    {
        // אני מוצאת את המיקום (האינדקס) של המוצר שצריך לעדכן
        List<Product> products = LoadProducts();
        int index = products.FindIndex(p => p.id == item.id);

        if (index == -1)
            throw new Exception($"Product with ID {item.id} not found");

        // בעיקרון, בשיטה הזו אני פשוט מחליפה את כל האובייקט הישן בחדש
        products[index] = item;
        SaveProducts(products);
    }

```

```
    public void Delete(int id)
    {
        // מוודאת שהמוצר באמת שם, ואז מעיפה אותו מהרשימה ושומרת
        List<Product> products = LoadProducts();
        Product? prod = products.FirstOrDefault(p => p.id == id);

        if (prod == null)
            throw new Exception($"Product with ID {id} not found");

        products.Remove(prod);
        SaveProducts(products);
    }
}

```